

AI Study Companion

AI-Powered Learning & Growth Workspace

ARCHITECTURE DOCUMENTATION

Candidate Challenge — Version 3.0

Document	Architecture Documentation
System	AI Study Companion — full-stack AI learning workspace
Frontend	Next.js 14 • TypeScript • Tailwind CSS • Recharts
Backend	FastAPI • Python • REST APIs • SSE
Database	PostgreSQL 16 + pgvector
Async Processing	Celery + Redis
AI	Gemini API + BGE-small embeddings
Core Pattern	Project-scoped grounded RAG + adaptive learning loop
Deployment	Vercel frontend + Render backend + managed data services

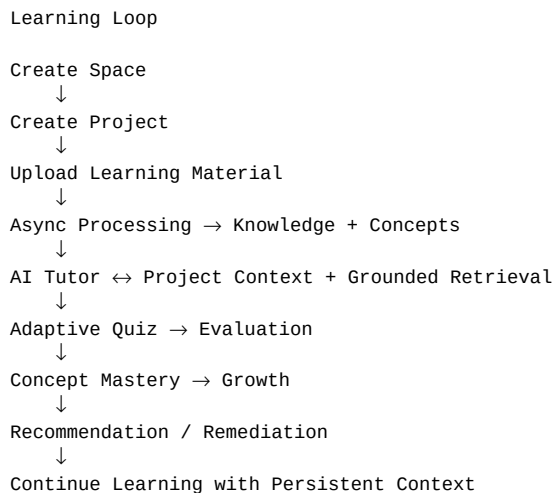
Document objective: explain the system architecture, component boundaries, data flows, AI/RAG design, assessment and mastery pipeline, security, observability, deployment, testing, and extensibility of the implemented application.

This document is based on the supplied project implementation reference and the architecture decisions established during development.

1. Executive Overview

The AI Study Companion is designed as a persistent learning workspace rather than a standalone chatbot. A learner creates a Space and one or more Projects, uploads learning material, studies with a project-grounded AI Tutor, completes adaptive assessments, receives concept-level mastery updates, sees growth insights, and receives an actionable next recommendation.

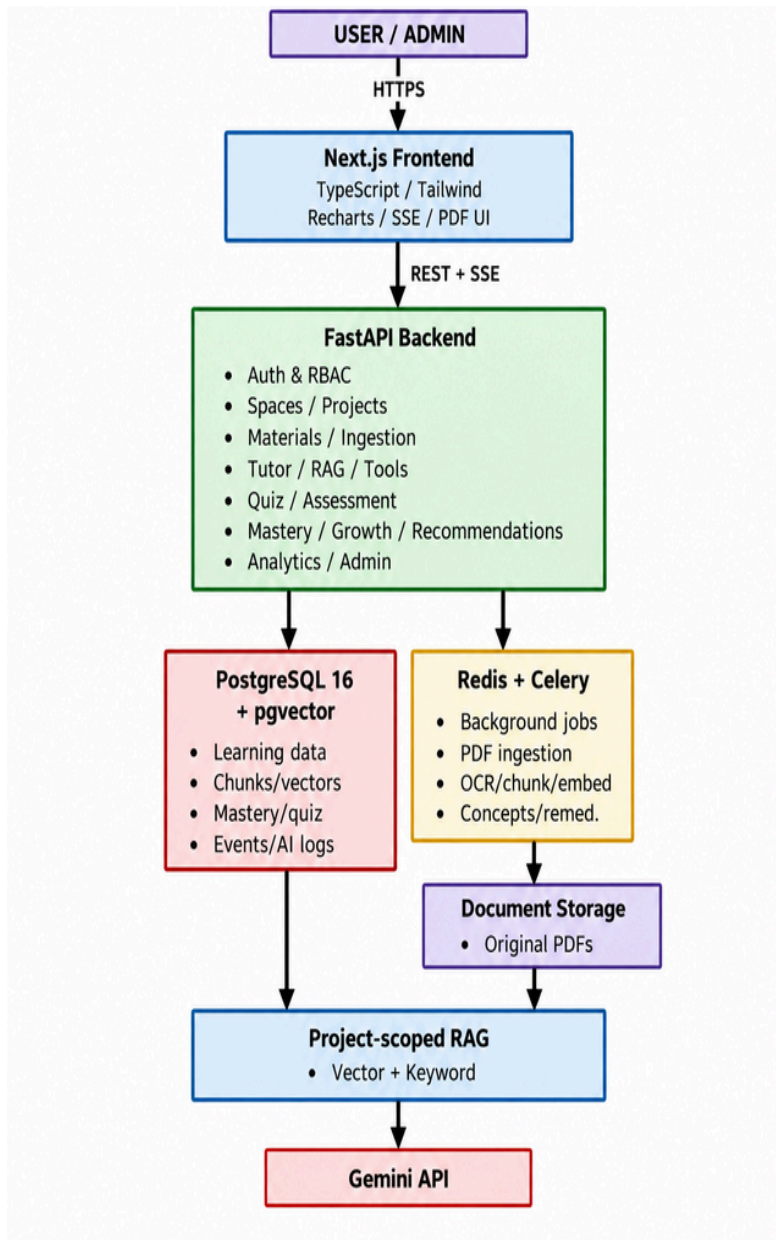
The architecture deliberately connects these capabilities into a closed learning loop. Knowledge produced from documents becomes retrieval context; Tutor and quiz interactions create evidence about understanding; that evidence updates mastery and learner context; and the resulting weaknesses and goals influence subsequent recommendations and remediation.



2. Architecture Goals

- Keep each Project as an isolated learning and knowledge boundary.
- Ground AI responses in uploaded evidence and handle insufficient evidence explicitly.
- Move expensive document processing to asynchronous workers.
- Make AI usage, failures, latency, retrieval quality and evaluation observable.
- Use structured, backend-validated AI actions instead of unrestricted AI access to application internals.
- Maintain persistent but relevant learner context across sessions.
- Keep the prototype deployable as a modular monolith while leaving clear extension points for scale.

3. High-Level System Architecture



The system uses a modular-monolith backend. The deployment model stays simple, while internal service boundaries separate learning, AI, assessment, analytics, administration, persistence and background processing.

4. Architectural Layers

Layer	Responsibility
Presentation	Next.js routes, components, forms, charts, Tutor streaming and PDF evidence viewer.
API/Application	FastAPI endpoints, authentication, authorization, request validation and response contracts.
Business Logic	Learning, Tutor, retrieval, quiz, assessment, mastery, recommendation and analytics services.
Data & Knowledge	PostgreSQL, pgvector, document chunks, learner context, events and original PDFs.
Background Processing	Celery/Redis workflows for document processing, embeddings, concept extraction and remediation.

Layer	Responsibility
AI/External Services	Gemini generation and BGE-small embeddings.
Observability/Evaluation	AI usage logs, latency/tokens/cost, retrieval metrics and curated evaluation suites.

5. Frontend Architecture

The frontend is implemented with Next.js 14 and TypeScript. The UI is organized around the learning hierarchy and exposes separate experiences for Spaces, Projects, materials, Tutor, quizzes, mastery/growth, analytics and administration.

- Authentication and protected application routes establish the active user session.
- Project pages provide the central context in which material, Tutor, quiz and progress features operate.
- The Tutor interface supports streamed responses through Server-Sent Events (SSE).
- Citation cards connect an answer to retrieved evidence and can open the authentic PDF at the cited page.
- The original PDF viewer preserves document fidelity instead of reconstructing uploaded material from extracted text.
- Recharts is used for mastery, progress, score and analytics visualizations.
- Structured Tutor tool results can be rendered as actionable UI widgets, such as mastery information or remedial-quiz actions.

6. Backend Modular Monolith

```
backend/app/
  api/v1/
    auth
    spaces
    projects
    materials
    tutor
    quiz / assessment
    admin
  core/
    configuration
    database
    JWT/security
  models/
  schemas/
  services/
    tutor
    retrieval
    citations
    remediation
    quiz
  ai/
    Gemini integration
    pricing / usage
    prompts
  workers/
    Celery/background workflows
```

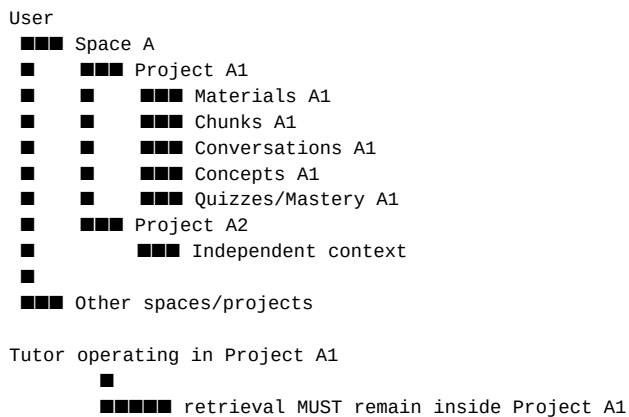
Pydantic schemas validate incoming and AI-generated structured data. Service modules contain domain behavior rather than placing learning logic directly inside API handlers.

7. Domain & Persistence Model

PostgreSQL is the primary system of record. pgvector stores document embeddings alongside normal relational learning data so authorization, learning state and semantic knowledge can be managed within the same persistence boundary.

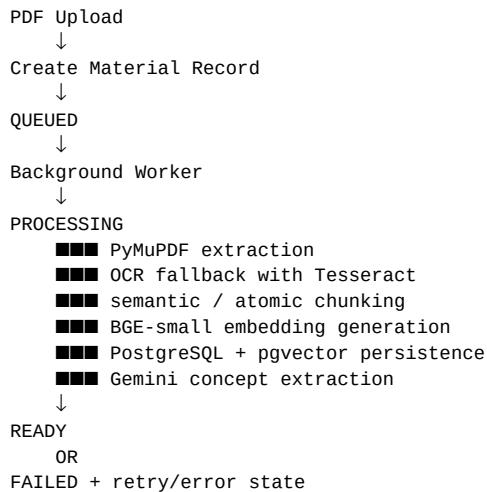
Entity / Area	Purpose
User	Identity, role and ownership.
Space	Higher-level learning organization.
Project	Focused learning goal and strict isolation boundary.
Material	Uploaded PDF, processing state and document metadata.
DocumentChunk	Chunk content, page number, project/material IDs, embedding and retrieval metadata.
Concept	Concepts extracted or associated with a Project.
ConceptMastery	Current estimated mastery and history/evidence.
Quiz / Question / Answer	Assessment state, generated questions and learner responses.
QuizMistake	Mistakes used for diagnosis and remediation.
LearnerContext	Goals, preferences, strengths, weaknesses and misconceptions.
ActivityEvent	Learning and system activity stream.
AI usage/evaluation	Model, feature, latency, token usage, estimated cost, errors and evaluation results.

8. Project as the Isolation Boundary



Authorization is performed before retrieval. Retrieval queries include the active Project boundary, so another Project's semantically similar content is not eligible as evidence. This rule applies to Tutor context and to learning state such as conversations, concepts, quiz history and mastery.

9. PDF Ingestion & Knowledge Processing



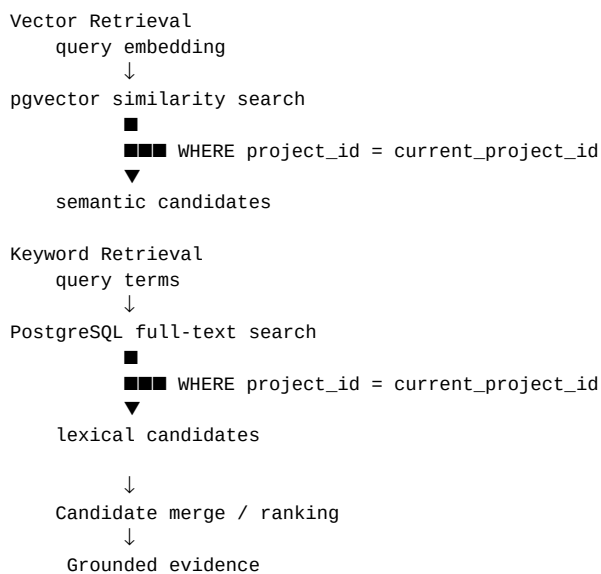
The processing pipeline is asynchronous. The request that accepts the upload does not need to wait for extraction, OCR, embedding and concept processing to finish.

9.1 Chunking and Metadata

- Chunks preserve `project_id` and `material_id` for isolation and traceability.
- Page number metadata is retained so evidence can be mapped back to the original PDF.
- Chunk index and content metadata support deterministic reconstruction and citation.
- Embeddings are generated using the selected BGE-small embedding model and stored in pgvector.
- Duplicate detection uses a document hash so repeated uploads do not unnecessarily repeat processing.

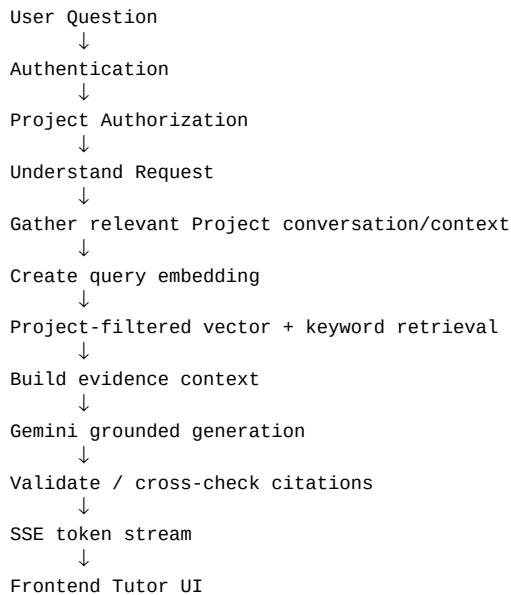
9.2 Retrieval Representation

The implemented architecture uses semantic vector retrieval through pgvector and supports keyword/full-text retrieval as a complementary signal. Candidate evidence can be merged before downstream context construction/reranking where enabled.



10. AI Tutor Architecture

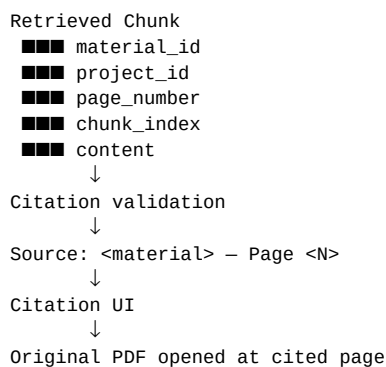
The Tutor is a project-aware conversational system. Its response is generated from the active Project's goal, retrieved learning evidence and relevant learner context rather than from unrestricted global application knowledge.



10.1 Grounding Contract

- Retrieved chunks are treated as evidence rather than optional hints.
- The Tutor is instructed not to fabricate facts that are unsupported by available Project material.
- When evidence is insufficient, the Tutor communicates that limitation instead of silently answering from unsupported knowledge.
- Source references are derived from actual retrieved metadata so that page citations can be checked.

10.2 Citation Architecture



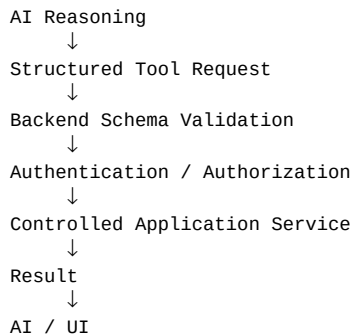
This makes the citation an evidence-traceability feature rather than merely text generated by the LLM.

11. Insufficient Evidence Handling

A dedicated unsupported-evidence path is important to the architecture. If a user asks about a topic that is not supported by the uploaded Project material, the system should state that the available evidence is insufficient. This is also a key evaluation dimension for the Tutor.

12. Structured AI Tool Interaction

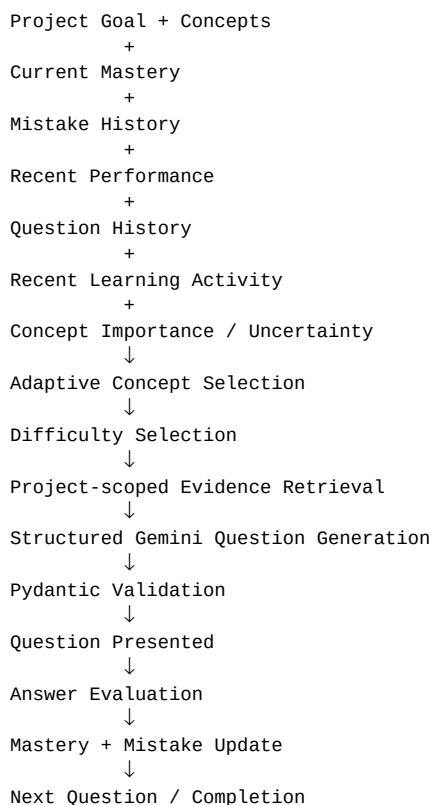
AI actions are controlled through an explicit tool boundary. The model does not receive unrestricted access to databases or internal services.



- **check_concept_mastery** — obtains project-scoped mastery information.
- **search_study_materials** — searches through the controlled retrieval layer.
- **trigger_remedial_quiz** — requests a targeted remediation assessment.
- **record_learner_preference** — stores an explicitly requested learner preference.

13. Adaptive Quiz & Assessment Architecture

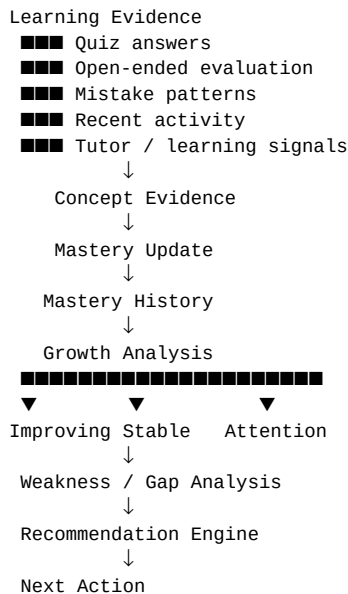
The adaptive quiz engine does not reduce adaptation to a single rule such as correct answer → harder or wrong answer → easier. It uses multiple learning signals.



13.1 Evaluation

- MCQ answers can be evaluated deterministically from the stored answer key.
- Open-ended answers are evaluated using understanding, accuracy, relevance/reasoning and missing-concept signals.
- Feedback separates demonstrated understanding from remaining gaps.
- Question history reduces unnecessary repetition and supports adaptive selection.

14. Mastery, Growth & Recommendation Architecture

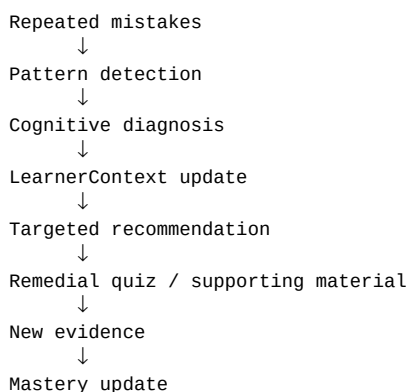


Mastery is treated as an evolving estimate supported by evidence rather than a fixed score adjustment. Historical values allow the system to reason about trajectories over time.

15. Persistent Learner Context

- Learning goals provide direction for future recommendations and assessments.
- Preferences can influence how learning interactions are presented.
- Strengths and weaknesses summarize accumulated evidence.
- Misconceptions provide targeted remediation context.
- Context is Project-aware so unrelated learning domains do not contaminate decisions.

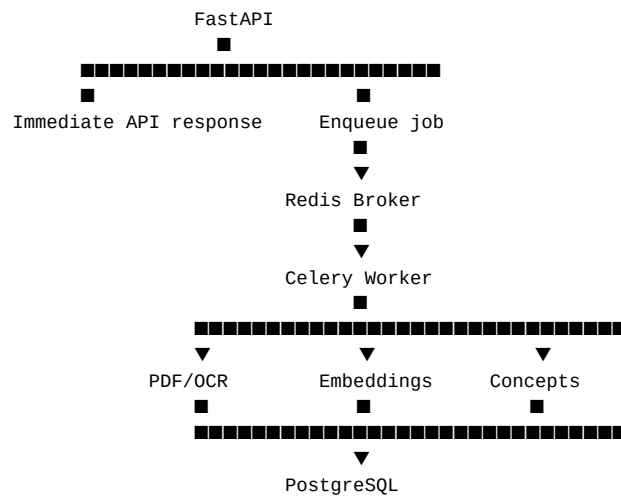
16. Repeated-Mistake Remediation



The remediation loop makes mistakes actionable rather than treating them only as historical statistics.

17. Background Processing Architecture

The system uses background processing for workflows that may involve document parsing, OCR, embeddings, concept extraction or chained learning updates.



18. Background Workflow Examples

- Material upload → ingestion → searchable knowledge → concept extraction → Project update.
- Quiz completion → answer evaluation → mastery update → weakness detection → insight → recommendation.
- Repeated mistakes → pattern detection → misconception diagnosis → learner-context update → targeted remediation.

18.1 Reliability Requirements

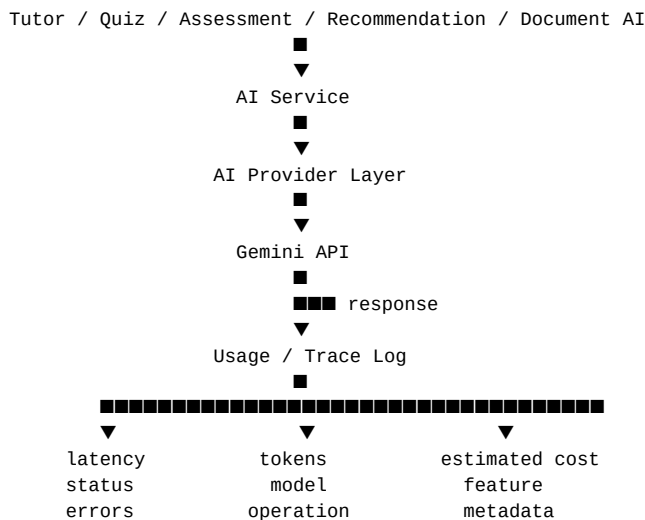
- Explicit material/job states make failures visible.
- Retryable failures use bounded retry behavior.
- Idempotent operations reduce duplicate side effects.
- Duplicate documents are detected before repeating expensive processing.
- Job ownership and Project authorization must be preserved when background work is executed.

19. Caching & Performance

- Redis can cache safe, repeatable reads without bypassing authorization.
- Bulk embedding and persistence reduce document-ingestion overhead.
- pgvector HNSW supports efficient approximate nearest-neighbor retrieval.
- Pagination is used for potentially large activity/analytics collections.
- SSE streams Tutor output incrementally to improve perceived latency.

20. AI Observability Architecture

AI is treated as an engineering subsystem. The architecture records enough operational metadata to answer questions such as which model was used, which feature was slow, how many tokens were consumed, what failed, and how retrieval quality changed.



21. AI Usage Metrics

Metric	Purpose
Model / Provider	Identify which AI implementation produced a result.
Feature / Operation	Separate Tutor, quiz, assessment, recommendation and document workloads.
Latency	Investigate slow AI responses.
Input / Output / Total Tokens	Understand AI usage when provider data is available.
Estimated Cost	Estimate consumption using configured pricing; not a billing statement.
Success / Failure	Measure reliability and classify errors.
Retrieval Metadata	Investigate poor context and source relevance.
Prompt / Model / Retrieval Version	Detect regressions after AI-system changes.

22. Evaluation Framework

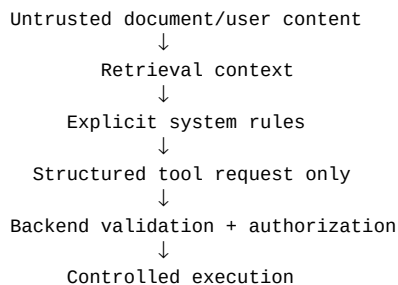
- Tutor: accuracy, groundedness, citation correctness and unsupported handling.
- Retrieval: relevance/source quality, Recall@K, Precision@K or MRR where curated ground truth exists.
- Assessment: question quality, grading quality, structured-output reliability and adaptive behavior.
- Recommendations: relevance, actionability and alignment with learner goals and weaknesses.
- Regression: compare evaluation runs after prompt, model or retrieval changes.

23. Security Architecture

- JWT authentication identifies the current user.
- Role-based guards separate learner and administrator operations.
- Project access is verified before Project-scoped data retrieval.
- Project IDs are carried into knowledge retrieval and learning-state queries to prevent cross-project leakage.
- AI-generated structured data is validated before persistence or state changes.
- Uploaded documents and user messages are treated as untrusted data rather than trusted instructions.
- Administrative global analytics are protected from ordinary learner access.
- Secrets are injected through environment variables and excluded from source control.

24. AI-Specific Security

The architecture treats prompt injection and untrusted document instructions as an application-security concern. Text retrieved from a document is evidence for answering a question, not authorization to execute an arbitrary operation.

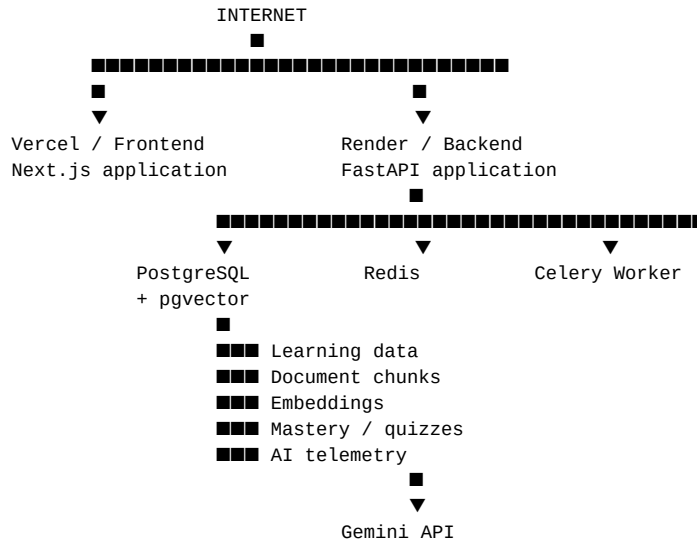


This prevents a retrieved document or user message from directly requesting database access, arbitrary internal-service calls, or unrestricted state changes.

25. Error Handling

- AI provider failures are classified rather than hidden.
- HTTP 429 rate-limit/quota failures use bounded exponential retry where appropriate.
- Timeouts prevent indefinitely blocked AI or background operations.
- Invalid structured AI responses are rejected and handled through controlled fallback/error paths.
- Material failures expose processing state and error information instead of falsely marking content as ready.
- Citation validation failures should not produce unsupported source claims.

26. Deployment Architecture



27. Production Configuration

- Frontend receives the deployed backend URL through environment configuration.
- Backend CORS allows the deployed frontend origin.
- Database connection strings, Redis credentials, JWT secrets and Gemini credentials are supplied as deployment secrets.
- The backend exposes health/API documentation endpoints for operational verification.
- The worker is deployed separately from the API process when using the Celery architecture.

28. Deployment Verification Checklist

- Frontend production build succeeds.
- Backend health endpoint responds.
- Authentication works on the deployed domain.
- PDF upload creates a background processing job.
- Material reaches READY state and becomes searchable.
- Tutor response streams successfully and citations resolve to the correct PDF pages.
- Adaptive quiz generation and answer evaluation work.
- Mastery, growth and recommendation pages reflect completed learning activity.
- Admin telemetry loads only for authorized administrators.
- Project A cannot retrieve Project B knowledge.

29. API Surface

Area	Representative Endpoint	Purpose
Authentication	POST /api/v1/auth/login	Authenticate and issue session/JWT.
Authentication	GET /api/v1/auth/me	Return current user.
Spaces	GET/POST /api/v1/spaces	List or create learning Spaces.
Projects	GET/POST /api/v1/spaces/{id}/projects	List or create Projects.
Materials	POST /api/v1/projects/{id}/materials	Upload PDF and enqueue processing.
Materials	GET /api/v1/projects/{id}/materials/{m_id}/raw	Return original PDF.
Tutor	POST /api/v1/projects/{id}/tutor/chat	Project-scoped Tutor/SSE interaction.
Quiz	POST /api/v1/projects/{id}/quizzes	Generate/start adaptive quiz.
Assessment	POST /api/v1/projects/{id}/quizzes/{q_id}/evaluate	Evaluate answers.
Mastery	GET /api/v1/projects/{id}/mastery	Project mastery/growth data.
Recommendations	GET /api/v1/projects/{id}/recommendations	Return next actions.
Analytics	GET /api/v1/projects/{id}/analytics	Project analytics.
Admin	GET /api/v1/admin/overview	Platform/admin overview.
Admin	GET /api/v1/admin/activity	Platform activity.
Admin	POST /api/v1/admin/evaluations/benchmark	Run evaluation benchmark.

30. End-to-End Learning Sequence

- 01 User authenticates
- 02 Creates Space
- 03 Creates Project + learning goal
- 04 Uploads PDF
- 05 Material enters QUEUED state
- 06 Worker extracts text / OCR
- 07 Content is chunked
- 08 BGE-small embeddings are generated
- 09 Chunks are stored in PostgreSQL + pgvector
- 10 Concepts are extracted
- 11 Material becomes READY
- 12 User asks Tutor question
- 13 Backend verifies Project access
- 14 Retrieval searches only Project knowledge
- 15 Gemini generates grounded response
- 16 Citation metadata is validated
- 17 Response streams to UI
- 18 User takes adaptive quiz
- 19 Answers are evaluated
- 20 Mastery / mistakes are updated
- 21 Growth is recalculated
- 22 Recommendation is generated
- 23 Activity + AI telemetry are recorded
- 24 Repeated mistakes can trigger remediation
- 25 Learner continues with retained relevant context

31. Repository Architecture

```
AI_PROF_ASSIGNMENT/
├── backend/
│   ├── app/
│   │   ├── api/v1/
│   │   ├── core/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── ai/
│   │   └── workers/
│   ├── evaluations/
│   ├── storage/
│   ├── tests/
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/
│   ├── src/app/
│   ├── src/components/
│   ├── src/lib/
│   ├── src/types/
│   └── package.json
├── docs/
├── docker-compose.yml
└── README.md
```

32. Testing Strategy

- Backend unit tests for services and business rules.
- Integration tests for API/database behavior.
- Security tests for Project isolation and authorization.
- RAG tests for retrieval, grounding and citation behavior.
- Adaptive assessment tests for concept/difficulty selection and evaluation.
- Frontend type checking and production-build verification.
- Curated AI evaluation datasets for Tutor, retrieval, assessment and recommendations.
- Regression tests after changes to prompts, models, retrieval or adaptive logic.

33. Representative Verification Commands

```
Backend
python -m pytest tests/ -v

Frontend
npx tsc --noEmit
npm run build

AI evaluation
python -m evaluations.runners.run_evaluations --type all
```

34. Key Architectural Decisions

Decision	Reason
Modular monolith	Simple deployment while maintaining internal domain boundaries.
Project as strict boundary	Prevents cross-project knowledge and learner-state contamination.
PostgreSQL + pgvector	Combines relational learning state and semantic knowledge retrieval.
Asynchronous ingestion	Prevents PDF processing from blocking API requests.

Decision	Reason
Grounded RAG	Keeps Tutor answers connected to uploaded evidence.
Backend citation verification	Prevents fabricated source/page references.
Structured tools	Controls AI actions through validation and authorization.
Persistent learner context	Allows evidence from prior learning to influence future decisions.
AI observability	Makes AI performance and usage measurable.

35. Scalability & Future Extensibility

The current modular-monolith architecture is appropriate for a prototype while preserving clear boundaries that can be independently scaled later.

- Scale document workers independently from API instances.
- Scale AI/evaluation workloads independently if model traffic increases.
- Use pgvector indexing and optimized retrieval as the knowledge corpus grows.
- Introduce a dedicated reranker when retrieval evaluation demonstrates a need.
- Add provider abstraction for multiple LLM and embedding providers.
- Move heavy evaluation jobs to dedicated worker queues.
- Add richer document understanding, multimodal inputs, voice and concept maps without changing the core Project boundary.
- Introduce spaced repetition, flashcards, study plans and notifications using existing concepts, mastery and learner context.

36. Architecture Risks & Mitigations

Risk	Mitigation
LLM hallucination	Grounded prompts, project-scoped retrieval, insufficient-evidence behavior and citation validation.
Prompt injection	Treat documents/user content as untrusted data and restrict tools through backend validation.
Cross-project leakage	Authorize before retrieval and apply project_id inside retrieval/state queries.
AI rate limits	Bounded retries, error classification and observability.
Slow document processing	Asynchronous workers with explicit job states.
Duplicate processing	Document hashing and idempotent jobs.
Bad structured AI output	Pydantic/schema validation before persistence.
AI regression	Curated evaluation suite and benchmark comparisons.
Growing activity data	Pagination, indexing and efficient analytics queries.

37. Final Architecture Summary

The AI Study Companion is architected as a project-isolated full-stack learning platform where AI participates in a measurable learning system rather than acting as an isolated chatbot. PostgreSQL and pgvector provide the persistent knowledge foundation; FastAPI provides the application and domain layer; Celery/Redis provide asynchronous execution; Gemini provides controlled generation; and BGE-small provides semantic embeddings.

The defining architecture is the closed learning loop: material becomes project knowledge, project knowledge grounds Tutor and assessment interactions, assessment evidence updates concept mastery, mastery and mistakes produce growth signals, and those signals drive recommendations and remediation. Project isolation, structured AI actions, citation verification, persistent learner context and AI observability provide the engineering controls around that loop.

The architecture therefore supports the core requirements of contextual learning, evidence-based AI, measurable growth, asynchronous processing, secure application interaction, and future extensibility while remaining practical to deploy as a modular monolith.

Submission note: This document is intended as the architecture deliverable accompanying the deployed AI Study Companion application.